



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Karina García Morales

Asignatura: Fundamentos de Programación

Grupo: 22

No de Práctica(s): Practica 8

Integrante(s): Roja Martínez Francisco Alexander

*No. de lista o
brigada:* 40

Semestre: 2026-1

Fecha de entrega: 28 de octubre de 2025

Observaciones:

CALIFICACIÓN:

Estructuras de repetición

Objetivo:

El alumno elaborará programas en C para la resolución de problemas básicos que incluyan las estructuras de repetición.

Introducción

Las estructuras de repetición, también llamadas estructuras cíclicas, iterativas o bucles, se utilizan para ejecutar una o varias instrucciones varias veces, siempre que se cumpla una condición lógica.

En el lenguaje C existen tres tipos principales de estructuras repetitivas: while, do-while y for. Las dos primeras, while y do-while, se consideran estructuras de uso general que permiten controlar ciclos de distintas formas según la necesidad del programa.

Licencia GPL de GNU

El software utilizado en esta práctica está protegido por la licencia GPL de GNU, lo que significa que es un programa libre: puede modificarse y compartirse libremente, siempre y cuando se mantenga la misma licencia.

La estructura repetitiva while

Evalúa primero una condición lógica; si esta se cumple (es verdadera), se ejecuta el bloque de instrucciones que está dentro de las llaves { }. Después de ejecutar el bloque, el programa vuelve a comprobar la condición, y mientras siga siendo verdadera, continuará repitiendo el mismo bloque de código. Cuando la condición deja de cumplirse (es falsa), el ciclo se detiene y el programa continúa con el flujo normal de ejecución.

Esto significa que el bloque de código dentro del while puede ejecutarse desde cero hasta varias veces, dependiendo del resultado de la condición.

Sintaxis:

```
while (expresión_lógica) {  
    // Bloque de código a repetir  
    // mientras que la expresión  
    // lógica sea verdadera.  
}
```

Estructura de control repetitiva do-while

La estructura do-while también permite repetir un bloque de código, pero a diferencia del while, primero ejecuta el bloque y después evalúa la condición. Esto garantiza que el código se ejecute al menos una vez, sin importar si la condición es verdadera o falsa al inicio.

Sintaxis:

```
do {  
    /*  
    Bloque de código que se ejecuta  
    por lo menos una vez y se repite  
    mientras la expresión lógica sea  
    verdadera.  
    */  
} while (expresión_lógica);
```

Estructura de control repetitiva for

El ciclo for se usa comúnmente cuando el número de repeticiones está controlado por una variable contador. Su estructura permite definir en una sola línea la inicialización de variables, la condición lógica y las operaciones que se realizan en cada iteración.

Sintaxis:

```
for (inicialización ; expresión_lógica ; operaciones por iteración) {  
    /*  
    Bloque de código  
    a ejecutar  
    */  
}
```

Directivas del preprocesador y #define

Las líneas de código que comienzan con el símbolo # son directivas del preprocesador, encargadas de realizar modificaciones en el código fuente antes de su compilación. Por ejemplo:

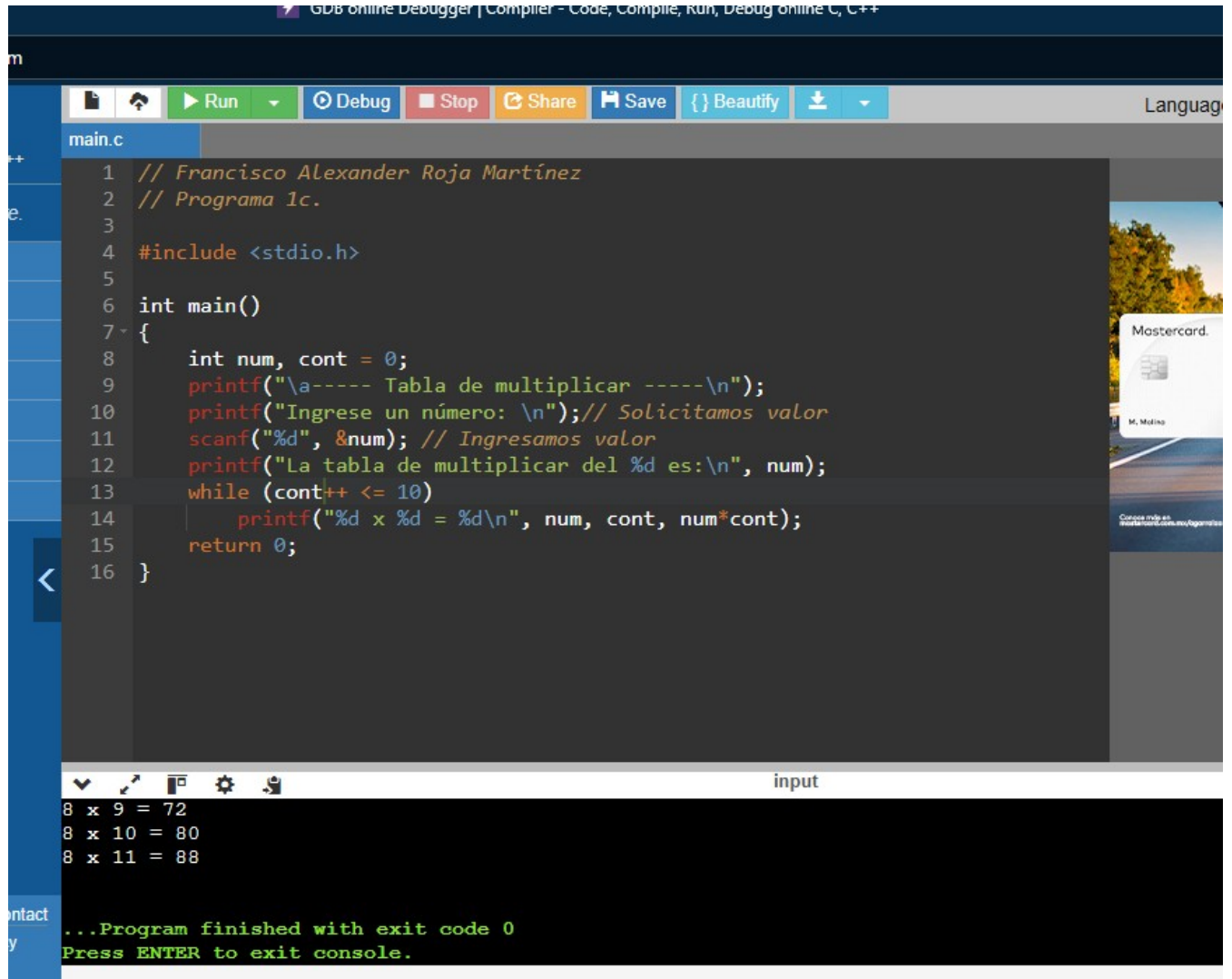
- #include sirve para **incluir archivos** o librerías.
- #define se usa para **definir constantes simbólicas**, que pueden representar un número o un texto.

Sintaxis: #define NOMBRE VALOR

Ejercicios en clase

1. Revisar el uso de ++cont <= 10 en el programa

Programa1.c



The screenshot shows a web-based GDB online debugger interface. At the top, there's a title bar "GDB online Debugger | Compiler - Code, Compile, Run, Debug online C, C++". Below it is a toolbar with buttons for Run, Debug, Stop, Share, Save, Beautify, and a download icon. The main area displays a C program named "main.c" with the following code:

```
1 // Francisco Alexander Roja Martínez
2 // Programa 1c.
3
4 #include <stdio.h>
5
6 int main()
7 {
8     int num, cont = 0;
9     printf("\a----- Tabla de multiplicar -----\\n");
10    printf("Ingrese un número: \\n"); // Solicitamos valor
11    scanf("%d", &num); // Ingresamos valor
12    printf("La tabla de multiplicar del %d es:\\n", num);
13    while (cont++ <= 10)
14    {
15        printf("%d x %d = %d\\n", num, cont, num*cont);
16    }
17    return 0;
18 }
```

At the bottom, there's a console window showing the program's output:

```
8 x 9 = 72
8 x 10 = 80
8 x 11 = 88
...Program finished with exit code 0
Press ENTER to exit console.
```

Esta sintaxis logra que la tabla de multiplicar se muestre **del 1 al 10**, incluyendo el num x 10, y es una forma concisa de controlar un bucle, combinando la inicialización del contador fuera, la condición, y el incremento dentro del propio while.

2. El uso de un break en el programa

Programa2.c



```
main.c
1 // Francisco Alexander Roja Martínez
2 // Programa 1c.
3
4 #include <stdio.h>
5 int main()
6 {
7
8     /* Al igual que en la estructura if-else, 0 -> falso y diferente de 0 -> verdadero.
9     El siguiente es un ciclo infinito porque la condición siempre es verdadera.
10    Así mismo, debido a que el ciclo consta de una sola línea, las llaves { }
11    opcionales.*/
12
13    while (100)
14    {
15        printf("Ciclo infinito.\nPara terminar el ciclo presione ctrl + c.\n");
16        break;
17    }
18    return 0;
19 }
```

input

Ciclo infinito.
Para terminar el ciclo presione ctrl + c.

...Program finished with exit code 0
Press ENTER to exit console.

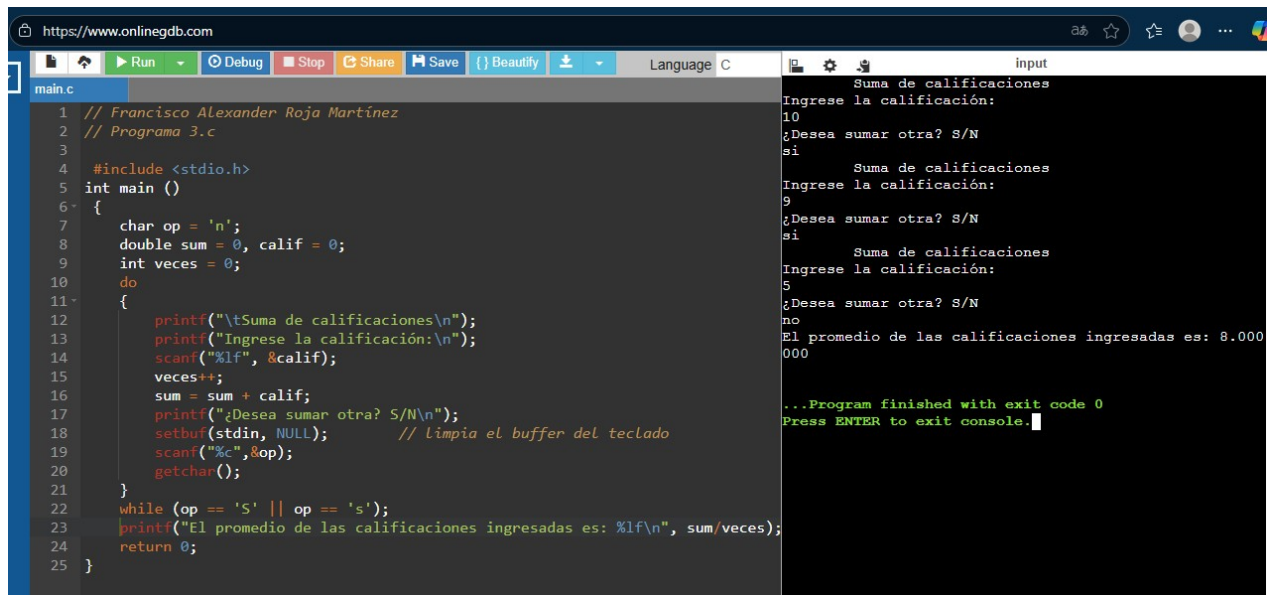
Bucle infinito: La condición `while(100)` crea un bucle infinito porque en C, cualquier valor numérico diferente de cero se evalúa como verdadero (100 es verdadero).

Uso de `break`: La sentencia `break` se utiliza para terminar inmediatamente la ejecución del bucle más interno en el que se encuentra. En este caso, el programa imprime el mensaje una vez y luego la sentencia `break` hace que el control del programa salga del bucle `while(100)` y continúe con la siguiente instrucción después del bucle, que es `return 0;`.

Pese a la descripción del comentario, el `break` en este ejemplo evita que se convierta en un bucle verdaderamente infinito al terminarlo en la primera iteración, demostrando su capacidad para finalizar un ciclo de forma controlada.

3. Para el programa programa3.c el uso de la limpieza de cache en teclado

Programa3.c



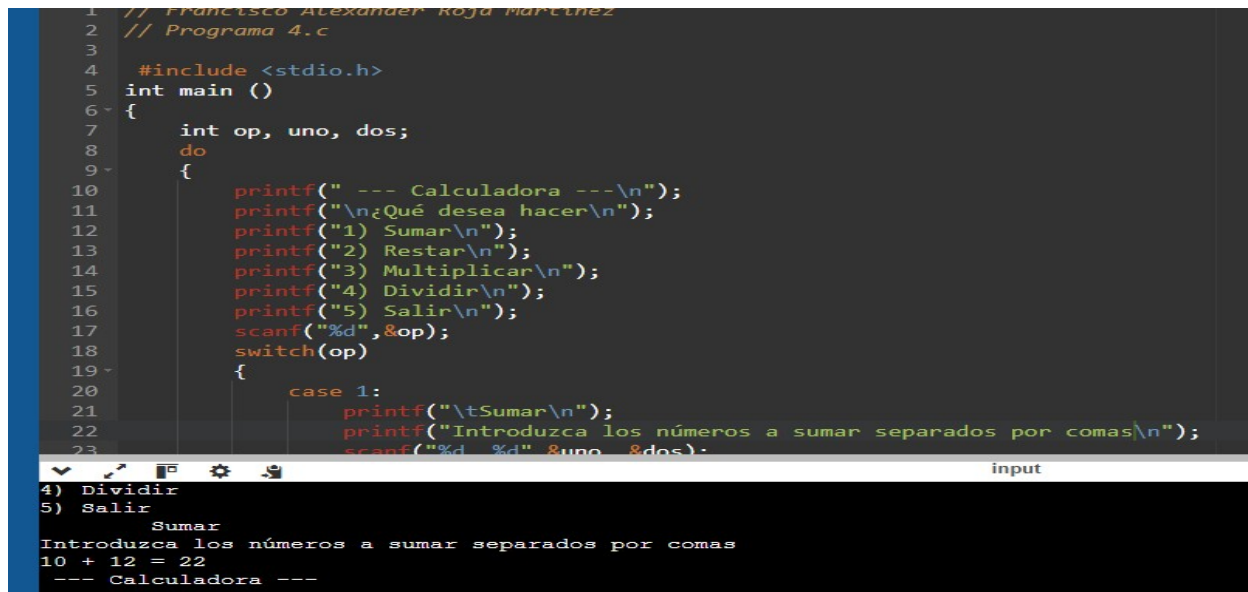
```
1 // Francisco Alexander Roja Martínez
2 // Programa 3.c
3
4 #include <stdio.h>
5 int main ()
6 {
7     char op = 'n';
8     double sum = 0, calif = 0;
9     int veces = 0;
10    do
11    {
12        printf("\tSuma de calificaciones\n");
13        printf("Ingrese la calificación:\n");
14        scanf("%lf", &calif);
15        veces++;
16        sum = sum + calif;
17        printf("¿Desea sumar otra? S/N\n");
18        setbuf(stdin, NULL); // Limpia el buffer del teclado
19        scanf("%c", &op);
20        getchar();
21    }
22    while (op == 'S' || op == 's');
23    printf("El promedio de las calificaciones ingresadas es: %lf\n", sum/veces);
24    return 0;
25 }
```

Suma de calificaciones
Ingrese la calificación:
10
¿Desea sumar otra? S/N
si
Suma de calificaciones
Ingrese la calificación:
9
¿Desea sumar otra? S/N
si
Suma de calificaciones
Ingrese la calificación:
5
¿Desea sumar otra? S/N
no
El promedio de las calificaciones ingresadas es: 8.000
000
...Program finished with exit code 0
Press ENTER to exit console.

La combinación de `setbuf(stdin, NULL)` y `getchar()` después de la lectura de caracteres asegura que el buffer del teclado esté limpio antes de la siguiente lectura de datos, garantizando que el programa espere la entrada correcta del usuario en cada iteración del bucle.

4. ¿Para el programa programa4.c que pasa si divido entre cero? Deberán agregar la condición.

Programa4.c

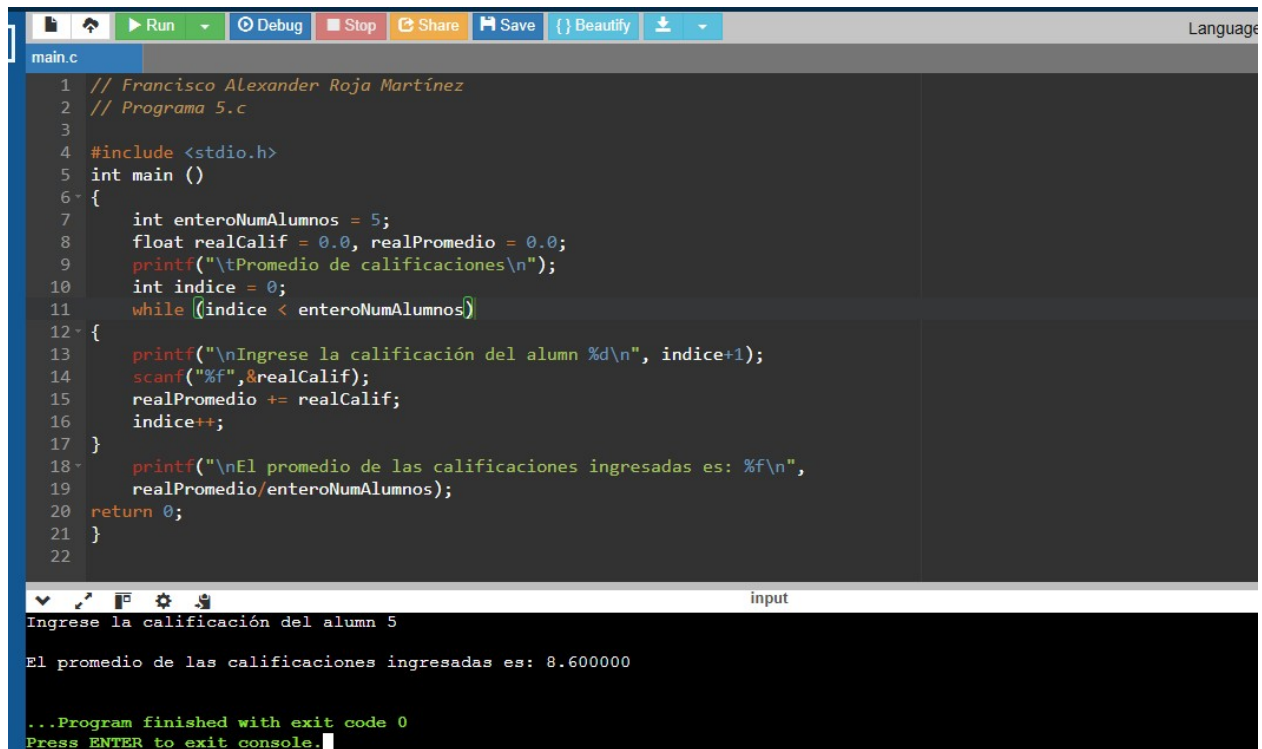


```
1 // Francisco Alexander Roja Martínez
2 // Programa 4.c
3
4 #include <stdio.h>
5 int main ()
6 {
7     int op, uno, dos;
8     do
9     {
10        printf(" --- Calculadora ---\n");
11        printf("\n¿Qué desea hacer\n");
12        printf("1) Sumar\n");
13        printf("2) Restar\n");
14        printf("3) Multiplicar\n");
15        printf("4) Dividir\n");
16        printf("5) Salir\n");
17        scanf("%d", &op);
18        switch(op)
19        {
20            case 1:
21                printf("\tSumar\n");
22                printf("Introduzca los números a sumar separados por comas\n");
23                scanf("%d %d", &uno, &dos);
```

4) Dividir
5) Salir
Sumar
Introduzca los números a sumar separados por comas
10 + 12 = 22
--- Calculadora ---

5. Modifica el programa programa5.c utilizando la estructura while

Programa5.c



```
main.c
1 // Francisco Alexander Roja Martínez
2 // Programa 5.c
3
4 #include <stdio.h>
5 int main ()
6 {
7     int enteroNumAlumnos = 5;
8     float realCalif = 0.0, realPromedio = 0.0;
9     printf("\tPromedio de calificaciones\n");
10    int indice = 0;
11    while (indice < enteroNumAlumnos)
12    {
13        printf("\nIngrese la calificación del alumn %d\n", indice+1);
14        scanf("%f",&realCalif);
15        realPromedio += realCalif;
16        indice++;
17    }
18    printf("\nEl promedio de las calificaciones ingresadas es: %f\n",
19        realPromedio/enteroNumAlumnos);
20    return 0;
21 }
22
```

input

Ingrese la calificación del alumn 5

El promedio de las calificaciones ingresadas es: 8.600000

...Program finished with exit code 0
Press ENTER to exit console.

El programa utiliza el bucle while para procesar un número fijo de calificaciones (definido por enteroNumAlumnos = 5).

Inicialización: La variable indice se inicializa a 0.

Condición de Continuación: El bucle while (indice < enteroNumAlumnos) se ejecuta mientras la variable indice sea menor que 5.

Cuerpo del Bucle:

Solicita e ingresa una calificación (realCalif).

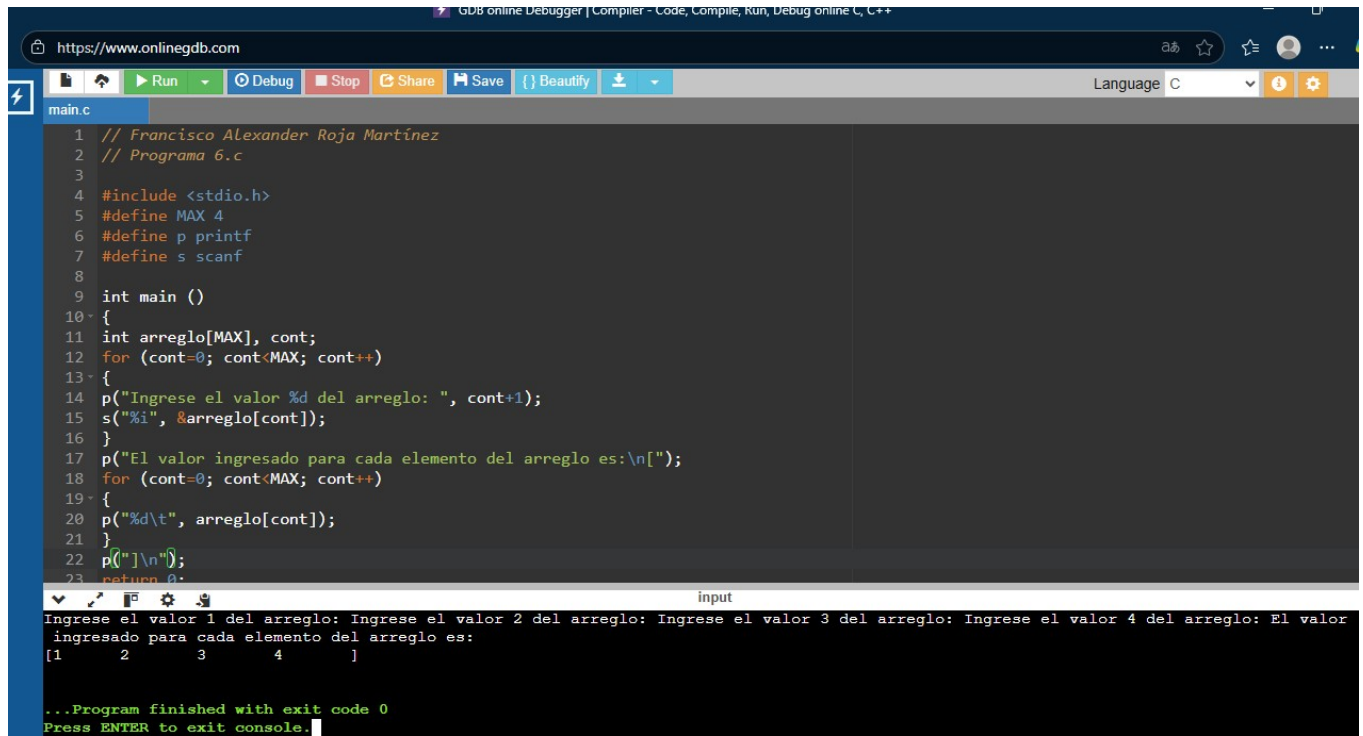
Suma la calificación a la variable acumuladora realPromedio.

Incrementa el contador de control (indice++).

Terminación: Cuando indice llega a 5, la condición se vuelve falsa, el bucle termina, y el programa procede a calcular e imprimir el promedio final fuera del bucle.

6. Después de analizar el ejercicio programa6.c indica para qué sirve la cláusula define

Programa6.c



```
1 // Francisco Alexander Roja Martínez
2 // Programa 6.c
3
4 #include <stdio.h>
5 #define MAX 4
6 #define p printf
7 #define s scanf
8
9 int main ()
10 {
11     int arreglo[MAX], cont;
12     for (cont=0; cont<MAX; cont++)
13     {
14         p("Ingrese el valor %d del arreglo: ", cont+1);
15         s("%i", &arreglo[cont]);
16     }
17     p("El valor ingresado para cada elemento del arreglo es:\n");
18     for (cont=0; cont<MAX; cont++)
19     {
20         p("%d\t", arreglo[cont]);
21     }
22     p("\n");
23     return 0;
24 }
```

input

Ingrese el valor 1 del arreglo: Ingrese el valor 2 del arreglo: Ingrese el valor 3 del arreglo: Ingrese el valor 4 del arreglo: El valor ingresado para cada elemento del arreglo es:

```
[1    2    3    4 ]
```

...Program finished with exit code 0
Press ENTER to exit console.

La cláusula #define es una directiva de preprocesador en C. El preprocesador es un programa que se ejecuta antes de que el código sea compilado.

La función principal de #define es crear macros.

Sustitución de Texto: La directiva #define le dice al preprocesador que reemplace cada aparición de un identificador (la macro) con un texto de reemplazo (su valor o cuerpo) en todo el código fuente antes de la compilación.

En el programa programa6.c, #define se utiliza con dos propósitos:

A. Definir Constantes

#define MAX 4: Define una constante simbólica llamada MAX. Cada vez que el preprocesador encuentre MAX, lo reemplazará con el valor 4. Esto es útil para:

Hacer el código más legible (es mejor usar MAX que el número 4).

Facilitar el mantenimiento (si se desea cambiar el tamaño del arreglo a 10, solo se modifica la línea del #define).

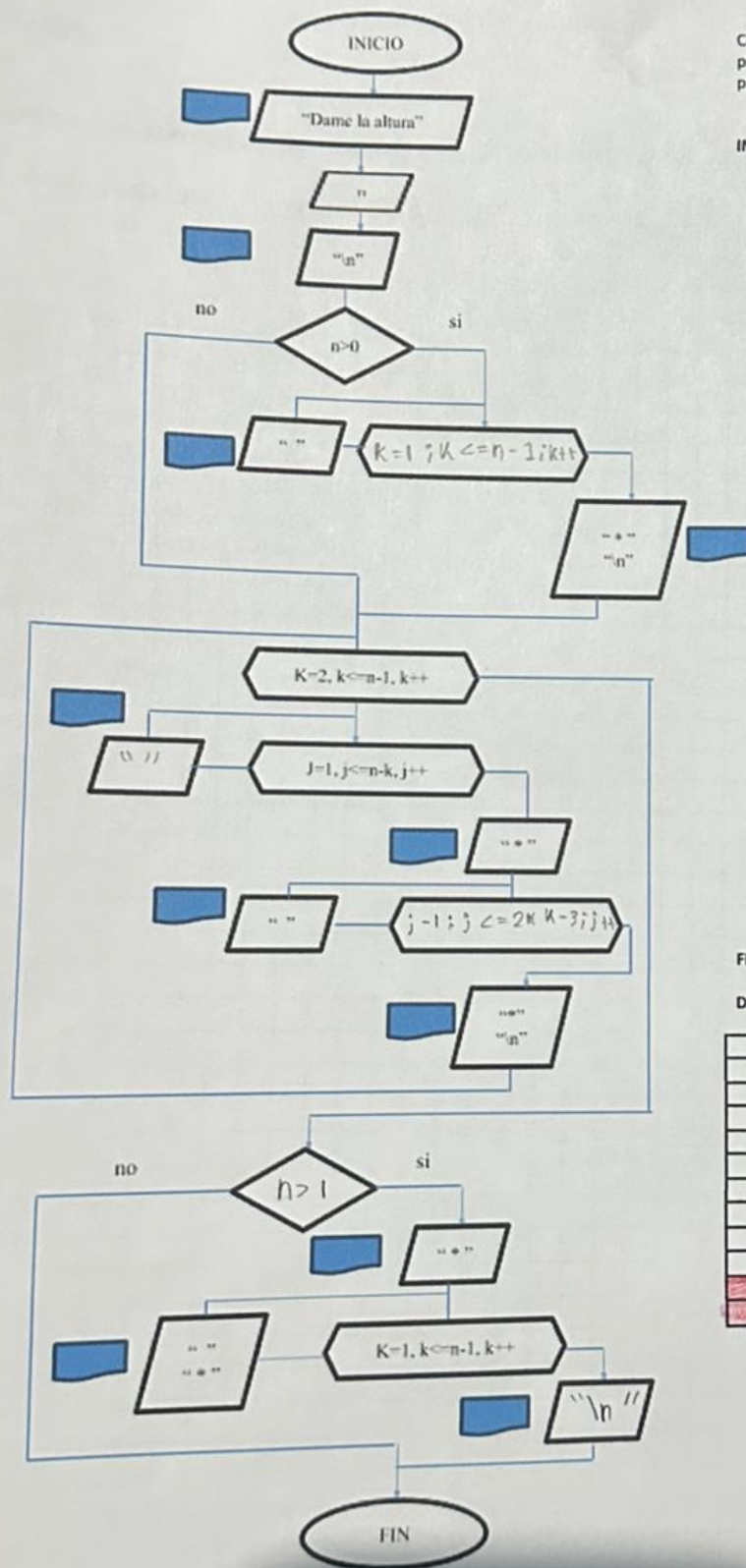
B. Definir Macros Similares a Funciones

`#define p printf`: Crea una macro para la función `printf`.

`#define s scanf`: Crea una macro para la función `scanf`.

El preprocesador reemplaza `p(...)` por `printf(...)` y `s(...)` por `scanf(...)` antes de que el compilador vea el código. Esto no crea nuevas funciones, sino que es un simple atajo o alias que puede acortar el código. Sin embargo, su uso debe ser cuidadoso, ya que las macros no tienen la misma verificación de tipos que las funciones regulares.

7.- Actividad: resuelve, codifica el problema y dibuja.



Compara y completa las instrucciones en el diagrama de flujo y el pseudocódigo, escribe en la cuadrícula cual es la salida de los algoritmos para un valor de $n=10$

```

INICIO
  ESCRIBIR "Dame la altura" ;
  LEER n;
  ESCRIBIR "\n";

  SI (n>0) ENTONCES
    PARA (k = 1; k <= n-1; k++)
      ESCRIBIR " ";
    FINPARA

    ESCRIBIR " ";
    ESCRIBIR "\n";
  FINSI

  PARA (K = 2; K <= n-1; K++)
    PARA (j = 1; j <= n-K; j++)
      ESCRIBIR " ";
    FINPARA

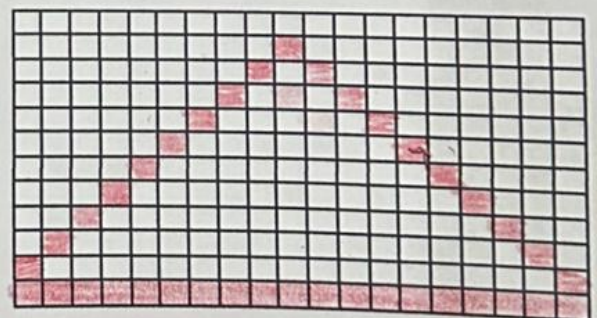
    ESCRIBIR " ";
    PARA (j = 1; j <= 2*K-3; j++)
      ESCRIBIR " ";
    FINPARA

    ESCRIBIR " ";
    ESCRIBIR "\n";
  FINPARA

  SI (n > 1) ENTONCES
    ESCRIBIR " ";
    PARA (k = 1; K <= 2*K-3; k++)
      ESCRIBIR " ";
    FINPARA

    ESCRIBIR "\n";
  FINSI
FIN
  
```

Dibuja la salida en el recuadro (primero identifica que escribe)



Tareas

1.- Solicitar radio a usuario y calcular el área.

```
1 // Roja Martínez Francisco Alexander
2
3 #include <stdio.h>
4 #define PI 3.14159
5
6 int main() {
7     float realRadio, realArea;
8
9     printf("--- Cálculo del Área de un Círculo ---\n");
10    printf("Ingrese el radio del círculo: ");
11    scanf("%f", &realRadio);
12
13    // Fórmula del área: Área = PI * radio * radio
14    realArea = PI * realRadio * realRadio;
15
16    printf("El área del círculo con radio %.2f es: %.2f\n", realRadio, realArea);
17
18    return 0;
19 }
20
```

input

--- Cálculo del Área de un Círculo ---
Ingrese el radio del círculo: El área del círculo con radio 3.00 es: 28.27

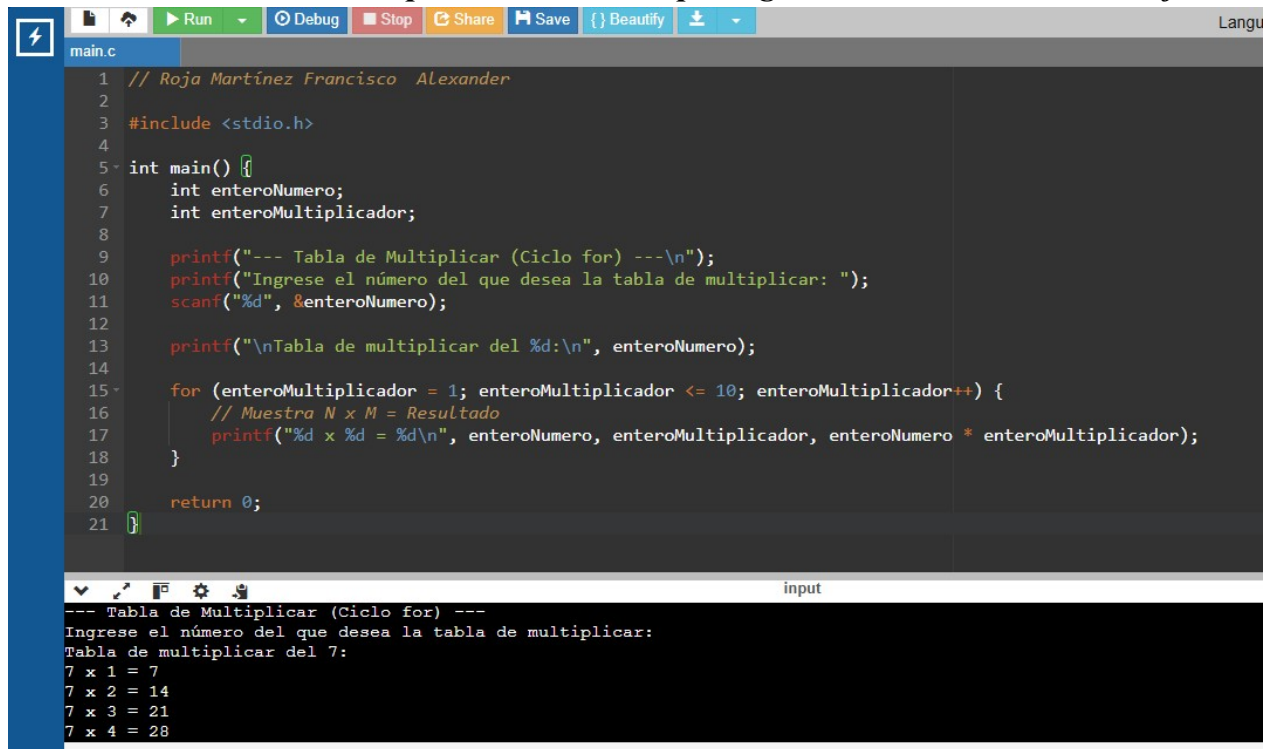
2.- Mostrar la numeración del 10 al 1 haciendo uso del ciclo while

```
1 // Roja Martínez Francisco Alexander
2
3 #include <stdio.h>
4
5 int main() {
6     int enteroContador = 10;
7
8     printf("--- Numeración del 10 al 1 (Ciclo while) ---\n");
9
10    while (enteroContador >= 1) {
11        printf("%d\n", enteroContador);
12        enteroContador--; // Decrementa el contador
13    }
14
15    return 0;
16 }
```

input

--- Numeración del 10 al 1 (Ciclo while) ---
10
9
8
7
6
5

3.- Mostrar la tabla de multiplicar del número que ingrese el usuario con el ciclo for.



```
1 // Roja Martínez Francisco Alexander
2
3 #include <stdio.h>
4
5 int main() {
6     int enteroNumero;
7     int enteroMultiplicador;
8
9     printf("--- Tabla de Multiplicar (Ciclo for) ---\n");
10    printf("Ingrese el número del que desea la tabla de multiplicar: ");
11    scanf("%d", &enteroNumero);
12
13    printf("\nTabla de multiplicar del %d:\n", enteroNumero);
14
15    for (enteroMultiplicador = 1; enteroMultiplicador <= 10; enteroMultiplicador++) {
16        // Muestra N x M = Resultado
17        printf("%d x %d = %d\n", enteroNumero, enteroMultiplicador, enteroNumero * enteroMultiplicador);
18    }
19
20    return 0;
21 }
```

input

```
--- Tabla de Multiplicar (Ciclo for) ---
Ingrese el número del que desea la tabla de multiplicar:
Tabla de multiplicar del 7:
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
```

4.- Genera un menú con los 3 ejercicios anteriores y agrega el ciclo do-while con el fin de que el usuario no pueda salir del programa hasta seleccionar la opción salir en el menú.

```
#include <stdio.h>
```

```
#define PI 3.14159 // Definición de PI para Tarea 1
```

```
// Prototipos de las funciones para las tareas
```

```
void tarea1_area_circulo();
```

```
void tarea2_conteo_while();
```

```
void tarea3_tabla_for();
```

```
int main() {
```

```
    int enteroOpcion;
```

```
do {

    printf("\n=====\\n");

    printf("      M E N Ú P R I N C I P A L\\n");

    printf("=====\\n");

    printf("1. Calcular el Área de un Círculo\\n");

    printf("2. Mostrar la numeración del 10 al 1\\n");

    printf("3. Mostrar la Tabla de Multiplicar\\n");

    printf("4. Salir del Programa\\n");

    printf("Seleccione una opción: ");

    scanf("%d", &enteroOpcion);


    switch (enteroOpcion) {

        case 1:

            tarea1_area_circulo();

            break;

        case 2:

            tarea2_conteo_while();

            break;

        case 3:

            tarea3_tabla_for();

            break;

        case 4:

            printf("¡Saliendo del programa! \\n");
```

```

        break;

    default:

        printf("Opción no válida. Intente de nuevo.\n");

        // Limpia el buffer de entrada para evitar ciclos infinitos si el usuario ingresa
algo no numérico

        while (getchar() != '\n');

        break;

    }

} while (enteroOpcion != 4); // Repite mientras la opción no sea 4 (Salir)

return 0;
}

// Implementación de la Tarea 1
void tarea1_area_circulo() {
    float realRadio, realArea;

    printf("\n--- Cálculo del Área de un Círculo ---\n");

    printf("Ingrese el radio del círculo: ");

    scanf("%f", &realRadio);

    realArea = PI * realRadio * realRadio;

    printf("El área del círculo con radio %.2f es: %.2f\n", realRadio, realArea);

}

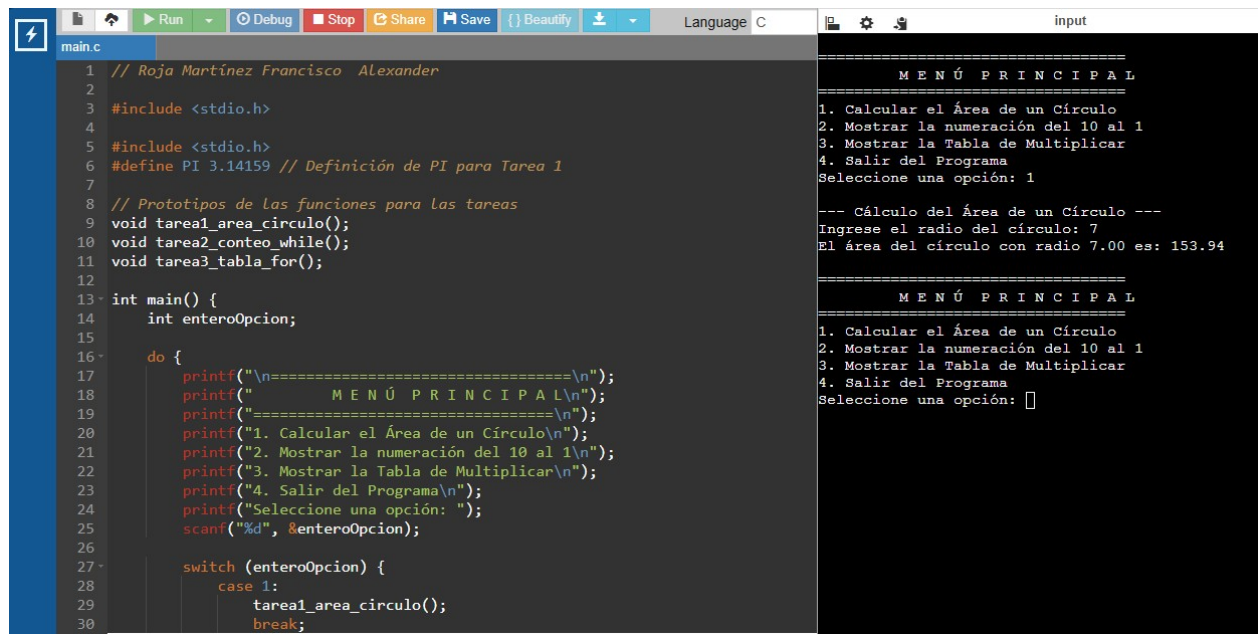
```

// Implementación de la Tarea 2

```
void tarea2_conteo_while() {  
    int enteroContador = 10;  
  
    printf("\n--- Numeración del 10 al 1 (Ciclo while) ---\n");  
  
    while (enteroContador >= 1) {  
        printf("%d\n", enteroContador);  
        enteroContador--;  
    }  
}
```

// Implementación de la Tarea 3

```
void tarea3_tabla_for() {  
    int enteroNumero;  
  
    int enteroMultiplicador;  
  
    printf("\n--- Tabla de Multiplicar (Ciclo for) ---\n");  
  
    printf("Ingrese el número del que desea la tabla de multiplicar: ");  
  
    scanf("%d", &enteroNumero);  
  
    printf("\nTabla de multiplicar del %d:\n", enteroNumero);  
  
    for (enteroMultiplicador = 1; enteroMultiplicador <= 10; enteroMultiplicador++) {  
        printf("%d x %d = %d\n", enteroNumero, enteroMultiplicador, enteroNumero *  
enteroMultiplicador);  
    }  
}
```

```
1 // Roja Martínez Francisco Alexander
2
3 #include <stdio.h>
4
5 #include <stdio.h>
6 #define PI 3.14159 // Definición de PI para Tarea 1
7
8 // Prototipos de Las funciones para las tareas
9 void tarea1_area_circulo();
10 void tarea2_conteo_while();
11 void tarea3_tabla_for();
12
13 int main() {
14     int enteroOpcion;
15
16     do {
17         printf("\n=====\\n");
18         printf("          M E N Ú   P R I N C I P A L\\n");
19         printf("=====\\n");
20         printf("1. Calcular el Área de un Círculo\\n");
21         printf("2. Mostrar la numeración del 10 al 1\\n");
22         printf("3. Mostrar la Tabla de Multiplicar\\n");
23         printf("4. Salir del Programa\\n");
24         printf("Seleccione una opción: ");
25         scanf("%d", &enteroOpcion);
26
27         switch (enteroOpcion) {
28             case 1:
29                 tarea1_area_circulo();
30                 break;
```

=====

M E N Ú P R I N C I P A L

=====

1. Calcular el Área de un Círculo
2. Mostrar la numeración del 10 al 1
3. Mostrar la Tabla de Multiplicar
4. Salir del Programa
Seleccione una opción: 1

--- Cálculo del Área de un Círculo ---
Ingrese el radio del círculo: 7
El área del círculo con radio 7.00 es: 153.94

=====

M E N Ú P R I N C I P A L

=====

1. Calcular el Área de un Círculo
2. Mostrar la numeración del 10 al 1
3. Mostrar la Tabla de Multiplicar
4. Salir del Programa
Seleccione una opción:

5.- Del programa5.c debe modificarlo para que permita solicitar al usuario el Número de alumnos a evaluar empleando un ciclo while en lugar del ciclo for, deben agregar lo que se pide al programa, no solo ejecutar.

```
#include <stdio.h>
```

```
int main() {
```

```
// Se elimina la inicialización a 5 y se inicializa a 0 para solicitarlo al usuario
```

```
int enteroNumAlumnos = 0;
```

```
float realCalif = 0.0, realPromedio = 0.0;
```

```
int indice = 0; // Se agrega un índice para el ciclo while
```

```
printf("\tPromedio de calificaciones\\n");
```

```
// 1. Solicitar al usuario el Número de alumnos a evaluar
```

```
printf("Ingrese el número de alumnos a evaluar: ");
```

```
scanf("%d", &enteroNumAlumnos);
```

```
// Validación básica para evitar división por cero
```

```
if (enteroNumAlumnos <= 0) {
```

```
    printf("El número de alumnos debe ser positivo. Saliendo.\n");
```

```
    return 1;
```

```
}
```

```
// 2. Usar un ciclo while en lugar del ciclo for
```

```
while (indice < enteroNumAlumnos) {
```

```
    printf("\nIngrese la calificación del alumno %d\n", indice + 1);
```

```
    scanf("%f", &realCalif);
```

```
    realPromedio += realCalif; // Acumula la calificación
```

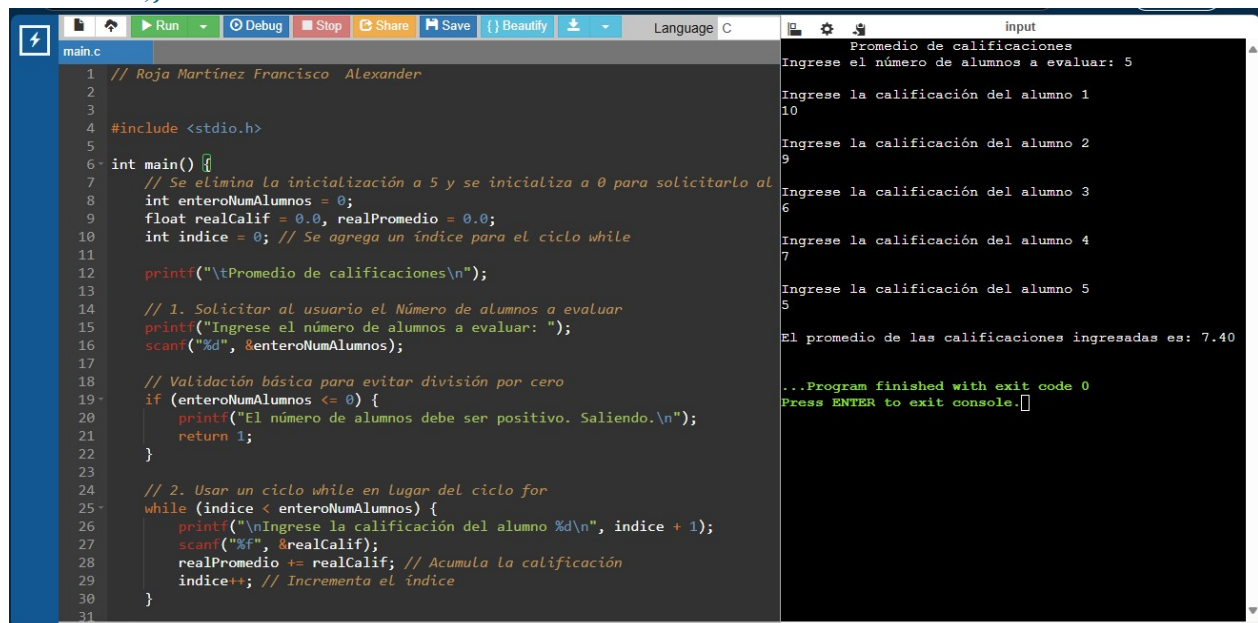
```
    indice++; // Incrementa el índice
```

```
}
```

```
printf("\nEl promedio de las calificaciones ingresadas es: %.2f\n",
```

```
    realPromedio / enteroNumAlumnos); // Se usa %.2f para un formato más limpio
```

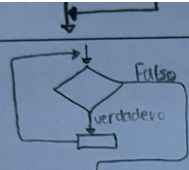
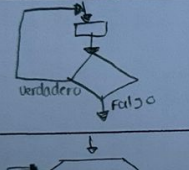
```
return 0;}
```



```
1 // Roja Martínez Francisco Alexander
2
3
4 #include <stdio.h>
5
6 int main() {
7     // Se elimina la inicialización a 5 y se inicializa a 0 para solicitarlo al
8     int enteroNumAlumnos = 0;
9     float realCalif = 0.0, realPromedio = 0.0;
10    int indice = 0; // Se agrega un índice para el ciclo while
11
12    printf("\tPromedio de calificaciones\n");
13
14    // 1. Solicitar al usuario el Número de alumnos a evaluar
15    printf("Ingrese el número de alumnos a evaluar: ");
16    scanf("%d", &enteroNumAlumnos);
17
18    // Validación básica para evitar división por cero
19    if (enteroNumAlumnos <= 0) {
20        printf("El número de alumnos debe ser positivo. Saliendo.\n");
21        return 1;
22    }
23
24    // 2. Usar un ciclo while en lugar del ciclo for
25    while (indice < enteroNumAlumnos) {
26        printf("\nIngrese la calificación del alumno %d\n", indice + 1);
27        scanf("%f", &realCalif);
28        realPromedio += realCalif; // Acumula la calificación
29        indice++; // Incrementa el índice
30    }
31}
```

Promedio de calificaciones
Ingrese el número de alumnos a evaluar: 5
Ingrese la calificación del alumno 1
10
Ingrese la calificación del alumno 2
9
Ingrese la calificación del alumno 3
6
Ingrese la calificación del alumno 4
7
Ingrese la calificación del alumno 5
5
El promedio de las calificaciones ingresadas es: 7.40
...Program finished with exit code 0
Press ENTER to exit console.

6.- Completa el cuadro con las estructuras iterativas (MIENTRAS, HACER MIENTRAS Y PARA), agrega solo el apartado en la tarea de la práctica.

Iterativa (repetitiva)	Mientras		Mientras (Condición) Hacer Instrucciones Fin- Mientras	<pre>int cont=0 while (cont<5){ printf("Hola\n"); cont++; } while (cont<5);</pre>
	Hacer-mientras		Hacer-mientras Instrucciones Mientras	<pre>int cont=0; do { printf("Hola\n"); cont++; } while (cont<5);</pre>
	Para		Para (Inicialización ; Condición Actualización) Hacer Instrucciones Fin-para	<pre>for (int i=0; i<5; i++) { printf("Hola\n"); }</pre>

Conclusión

Esta práctica me resultó más sencilla que la mayoría. Gracias a la buena comprensión que adquirí en clase sobre los ciclos (for, while, do-while), la implementación de los ejercicios fue mucho más fluida y lógica.

Aunque en general fue fácil, tuve una duda específica (particularmente con la gestión del buffer en el menú do-while de la Tarea 4). Para resolverla y garantizar un código robusto, recurrí a la ayuda de Internet y a la Inteligencia Artificial, tal como lo he hecho en prácticas anteriores.

Me gustó mucho esta práctica, ya que me permitió aplicar y consolidar directamente los conceptos aprendidos, confirmando que una buena base en clase facilita enormemente la codificación.

Referencia

Facultad de Ingeniería. (2025). Manual de prácticas del laboratorio de Fundamentos de Programación. Laboratorio de computación. Salas A y B. Universidad Nacional Autónoma de México. pp.126-134 . Recuperado el 21 de octubre de 2025 de [Laboratorio Salas A y B](#)

